System Design Interview Questions and Key Technical Areas

A structured preparation guide: common interview problems, reusable engineering concepts, and a practical study sequence.

How to use this guide
Do not memorize every design independently. Learn the recurring patterns—caching, partitioning, replication, consistency, queues, concurrency, and fault tolerance—and apply them to different systems.

Contents

• 1. System design interview question bank

• 2. Fundamental technical areas

• 3. Recommended preparation order

• 4. A reusable interview framework

• 5. Final checklist

1. System Design Interview Question Bank

A. Core system design problems

Question	Key technical areas
URL Shortener (TinyURL)	Unique ID generation, hashing, database indexes, caching, redirects, read-heavy workloads.
Rate Limiter	Token bucket, sliding window, Redis, atomic operations, distributed counters.
API Gateway	Routing, authentication, throttling, load balancing, service discovery, resilience.
Notification System	Queues, retries, delivery guarantees, deduplication, scheduling, dead-letter queues.

B. Application and social platforms

Question	Key technical areas
WhatsApp / Chat	WebSockets, connection management, message ordering, delivery acknowledgements, offline messages.
Twitter / News Feed	Fan-out on write/read, timelines, caching, ranking, hot users.
Instagram	Object storage, metadata, media processing, CDN, feed generation.
YouTube / Video Streaming	Uploads, transcoding, object storage, CDN, adaptive streaming.
Google Drive / Dropbox	Chunking, synchronization, versioning, metadata, conflict resolution.
Search Autocomplete	Trie, prefix search, ranking, caching, index updates.
Web Crawler	Distributed task queues, URL deduplication, politeness, scheduling.
Recommendation System	Event collection, feature storage, ranking, offline and online processing.

1. System Design Interview Question Bank (continued)

C. Distributed systems and data infrastructure

Question	Key technical areas
Kafka	Partitioning, consumer groups, offsets, replication, ordering, retention.
Distributed Cache	Consistent hashing, eviction, replication, cache invalidation, hot keys.
Distributed Key-Value Store	Partitioning, replication, quorum, consistency, failure recovery.
Distributed ID Generator	Snowflake-style IDs, time, worker IDs, clock issues, uniqueness.
CDC Platform	Transaction logs, change capture, ordering, checkpoints, replay.
Data Lake	Object storage, ingestion, partitioning, file formats, catalog, compaction.
Metrics Monitoring System	Time-series storage, aggregation, downsampling, alerting, retention.
Log Aggregation Platform	Streaming ingestion, indexing, search, retention, backpressure.

D. Transactions, booking and real-time systems

Question	Key technical areas
Ticket Booking	Inventory reservation, race conditions, locking, expiration, idempotency.
Payment System	State machines, idempotency, ledger, reconciliation, retries.
E-commerce Platform	Catalog, cart, inventory, order processing, transactions.
Uber / Ride Sharing	Geospatial indexing, nearby drivers, matching, location updates.
Google Maps	Road graphs, shortest paths, geospatial indexing, routing.
Distributed Task Scheduler	Scheduling, leases, retries, worker coordination, duplicate execution.
Food Delivery System	Matching, tracking, state transitions, event-driven workflows.

E. Modern AI and GenAI systems

Question	Key technical areas
RAG Platform	Document ingestion, chunking, embeddings, vector search, reranking.
Vector Database	ANN indexes, metadata filters, sharding, replication, recall vs latency.
LLM Gateway	Routing, rate limits, quotas, caching, fallbacks, token budgets.
AI Agent Platform	Tool execution, workflow state, checkpoints, permissions, retries.
LLM Evaluation Platform	Test datasets, evaluation metrics, tracing, experiments, regression testing.

2. Fundamental Technical Areas

1. Networking and communication

HTTP/HTTPS, DNS, TCP, load balancers, reverse proxies, WebSockets, connection pooling, timeouts.

2. Databases and data modeling

SQL vs NoSQL, indexes, transactions, isolation levels, query patterns, schema design, partitioning, replication.

3. Caching

Cache-aside, write-through, TTL, eviction, invalidation, cache stampede, hot keys, distributed caches.

4. Distributed systems fundamentals

CAP, consistency models, quorum, leader election, consensus basics, replication lag, split-brain, idempotency.

5. Messaging and event-driven architecture

Kafka, queues, pub/sub, ordering, at-least-once delivery, deduplication, outbox pattern, backpressure.

6. Reliability and fault tolerance

Retries, exponential backoff, circuit breakers, bulkheads, failover, graceful degradation, disaster recovery.

7. Scalability and performance

Horizontal scaling, sharding, replication, throughput, latency, capacity estimation, bottleneck analysis.

8. Observability and operations

Logs, metrics, distributed tracing, SLOs, alerting, deployment strategies, health checks, capacity planning.

9. Concurrency

Race conditions, locks, optimistic concurrency, atomic operations, leases, deadlocks, concurrent updates.

10. Security

Authentication, authorization, TLS, secrets management, least privilege, abuse prevention, data protection.

11. Storage and content delivery

Object storage, CDNs, caching layers, file uploads, lifecycle policies, durability.

12. Cost optimization

Storage tiers, workload sizing, autoscaling, data retention, caching trade-offs, managed vs self-managed services.

3. Recommended Preparation Order

Order	Topic	Practice question
1	Requirements, APIs, data modeling	URL Shortener
2	Indexes, caching, read/write patterns	Distributed Cache
3	Hashing and partitioning	Key-Value Store
4	Queues, events and retries	Notification System
5	Concurrency and consistency	Ticket Booking
6	Replication and fault tolerance	Kafka
7	Real-time communication	Chat System
8	Geospatial systems	Nearby Drivers
9	Data-intensive pipelines	Data Lake / CDC
10	AI infrastructure	RAG Platform / LLM Gateway

4. Reusable System Design Interview Framework

1. Clarify requirements

Identify functional requirements, scale, constraints, users, and non-functional requirements.

2. Estimate capacity

Estimate requests per second, storage, bandwidth, peak traffic, and growth.

3. Define APIs and data model

Specify important endpoints, entities, access patterns, and database indexes.

4. Draw the high-level architecture

Show clients, services, databases, caches, queues, and external dependencies.

5. Deep dive into the hardest component

Choose the key challenge: consistency, ordering, fan-out, search, matching, or throughput.

6. Handle failures and concurrency

Discuss retries, duplicates, race conditions, timeouts, failover, and recovery.

7. Explain trade-offs

Compare consistency vs availability, latency vs cost, and simplicity vs scalability.

8. Cover operations

Include monitoring, security, deployment, alerting, disaster recovery, and capacity growth.

5. Final Checklist

- ■ Can I state the functional and non-functional requirements clearly?
- ■ Can I estimate traffic, storage, bandwidth, and peak load?
- ■ Can I explain why I chose a database and its data model?
- ■ Can I identify bottlenecks and explain how the system scales?
- ■ Can I explain consistency, replication, partitioning, and failure recovery?
- ■ Can I handle retries, duplicate requests, race conditions, and idempotency?
- ■ Can I explain monitoring, security, and operational trade-offs?
- ■ Can I defend the design and explain what I would change at 10x scale?

Final principle: The objective is not to design the most complicated system. Explain why each component exists, what guarantees it provides, and which trade-offs it introduces.